Hotel Booking Database System Project.

Student Name: _Leon Kimutai Langat_

Student ID: _GH1046169_

Course: B103B Databases & Big Data Structure.

Submission Date: _08/04/2026_

GitHub Repository Link: _____

Video Demonstration Link: Screen Recording 2026-04-08 232257

1.Introduction.

This project develops a hotel booking system to manage reservations, room availability, and payments efficiently. The system uses a relational database to store data in an organized way, making it easy to access, update, and maintain.

A relational database is suitable because it reduces data redundancy, ensures consistency, and allows smooth connections between different parts of the system. The database includes four main tables: **Customers** (personal details), **Rooms** (room types and prices), **Bookings** (links customers to rooms with reservation dates), and **Payments** (tracks transactions).

Sample data is added to demonstrate real-world usage, and SQL scripts are used to create and populate the tables. Overall, the system is designed to be clear, efficient, and reliable for managing hotel booking operations.

2. System Design

2.1 Overview

The hotel booking database is designed to manage reservations, room availability, customer information, and payments in an organized and efficient way. The system ensures that all data is stored systematically, making it easy to access, update, and maintain. By using a relational database, the system supports smooth data flow between different parts of the application while minimizing errors and redundancy

2.2 Entities

The system is composed of four main entities: Customers, Rooms, Bookings, and Payments. Each entity represents a real-world object and is implemented as a table in the database.

Customers: Stores details such as name, email, and phone number for individuals making bookings. This allows the system to track who is using the hotel services.

Rooms: Contains information about each room, including type (single, double, suite), price, and current availability. This helps in managing room allocation effectively.

Bookings: Acts as a link between customers and rooms, recording which customer booked which room and for what period. It ensures accurate tracking of reservations.

Payments: Tracks financial transactions for each booking, including amount and payment date, ensuring proper accounting and billing.

2.3 Relationships

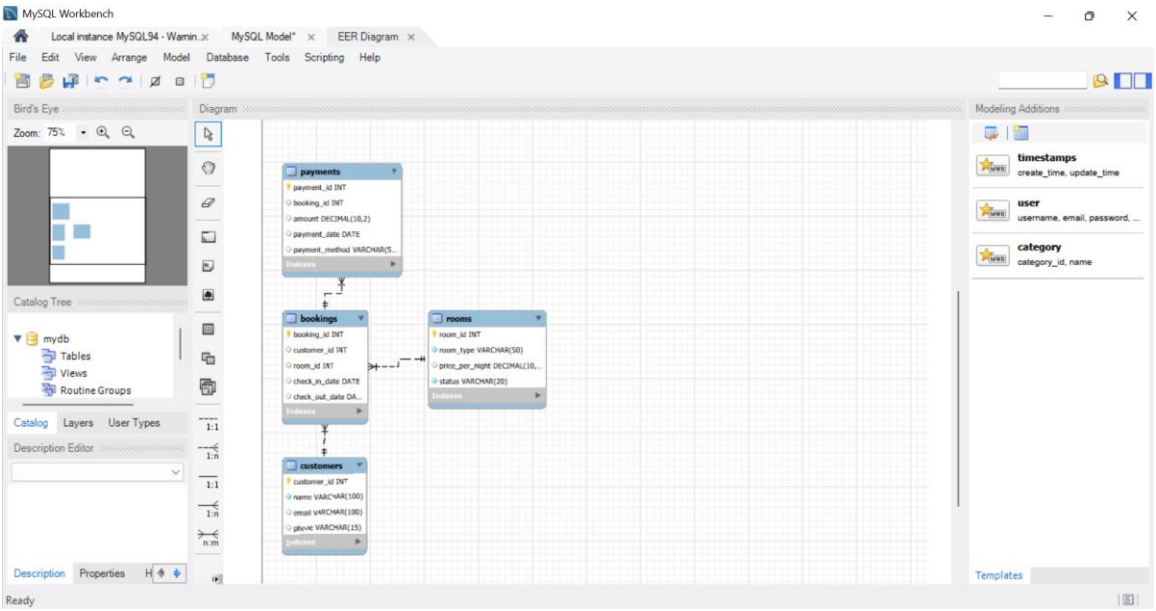
The relationships between the entities are essential for maintaining the integrity of the database.

- A one-to-many relationship exists between Customers and Bookings, as one customer can make multiple bookings, but each booking is associated with only one customer.
- A one-to-many relationship also exists between Rooms and Bookings. A single room can be booked multiple times over different periods, but each booking refers to one specific room.
- A one-to-one relationship exists between Bookings and Payments, where each booking has a corresponding payment record.

These relationships are implemented using foreign keys, which ensure referential integrity and prevent invalid data entries. The use of these relationships allows the database to accurately represent real-world hotel booking interactions.

2.4 ER Diagram

Insert your ER diagram here and explain it.



The Entity-Relationship (ER) diagram provides a visual representation of the database structure. It illustrates the entities, their attributes, and the relationships between them.

In the ER diagram, each entity is represented as a rectangle, while attributes are listed within or connected to the entity. Primary keys are clearly indicated, and foreign keys are

used to establish relationships between entities. Lines connecting the entities represent relationships, with cardinality (one-to-many, one-to-one, many-to-many) clearly defined.

The diagram shows that the Bookings entity acts as a linking table between Customers and Rooms, while the Payments entity is directly connected to Bookings. This visual representation helps in understanding how data flows within the system and ensures that the database design is logically structured.

2.5 Normalization
Explain 1NF, 2NF, 3NF and how your design follows them.

Normalization is applied in the database design to eliminate redundancy and ensure data integrity. The design follows the first three normal forms:

- **First Normal Form (1NF):** All tables contain atomic values, meaning that each field holds only a single value and there are no repeating groups. For example, customer details are stored in separate fields such as name and email.
- **Second Normal Form (2NF):** The database eliminates partial dependencies by ensuring that all non-key attributes depend entirely on the primary key. Each table has a primary key, and all attributes are directly related to it.
- **Third Normal Form (3NF):** The design removes transitive dependencies, meaning that non-key attributes do not depend on other non-key attributes. For example, payment details are stored separately in the Payments table rather than being included in the Bookings table.

By applying normalization, the database achieves a clear and efficient structure, reduces duplication of data, and improves consistency. This ensures that the system performs reliably and can be easily maintained or expanded in the future.

3. Implementation

3.1 Database Environment
Mention DBMS used.

The database for the hotel booking system was implemented using a relational database management system (RDBMS). In this project, a system, MySQL was selected due to its reliability, ease of use, and strong support for SQL operations.

The chosen DBMS provides essential features such as data integrity, transaction management, and support for complex queries. It also allows for the enforcement of constraints and relationships between tables, which are critical for maintaining a consistent and accurate database.

Additionally, the database was developed and tested using a graphical interface tool (such as MySQL Workbench), which simplifies the process of writing and executing SQL queries, as well as visualizing the database structure.

3.2 Table Creation

The database consists of four main tables: Customers, Rooms, Bookings, and Payments. Each table was carefully designed to represent a specific part of the hotel booking system.

Customers Table

The Customers' table is used to store information about individuals who make bookings. It includes attributes such as customer ID, name, email, and phone number. The customer ID is defined as the primary key to ensure that each record is unique. Additional constraints such as NOT NULL are applied to important fields to ensure that essential information is always provided.

Rooms Table

The Rooms table stores details about the hotel rooms. This includes attributes such as room ID, room type, price per night, and status. The room ID serves as the primary key. The price attribute is designed to store numeric values, and constraints are applied to ensure that only valid values are entered. The status field is used to indicate whether a room is available or occupied.

Bookings Table

The Bookings table is the central component of the system, linking customers and rooms. It includes attributes such as booking ID, customer ID, room ID, check-in date, and check-out date. The booking ID is the primary key, while customer ID and room ID are foreign keys that reference the Customers and Rooms tables respectively.

This table ensures that each booking is associated with a valid customer and room, and it records the duration of each stay.

Payments Table

The Payments table is used to record financial transactions related to bookings. It includes attributes such as payment ID, booking ID, amount, payment date, and payment method. The payment ID is the primary key, and the booking ID acts as a foreign key linking each payment to a specific booking.

This structure ensures that all payments are properly tracked and associated with the correct reservation.

3.3 Constraints

Constraints are applied in the database to maintain data integrity and enforce rules on the data being stored.

- **Primary Key (PK):** Each table includes a primary key that uniquely identifies each record. For example, customer_id in the Customers table and booking_id in the Bookings table ensure that no duplicate records exist.
- **Foreign Key (FK):** Foreign keys are used to establish relationships between tables. For instance, the customer_id in the Bookings table references the Customers table, ensuring that a booking cannot exist without a valid customer.
- **NOT NULL:** This constraint is used to ensure that certain fields cannot be left empty. Important attributes such as customer name, room type, and booking dates are required to maintain meaningful records.
- **CHECK:** The CHECK constraint is used to enforce specific conditions on data values. For example, the price per night must be greater than zero, ensuring that invalid or unrealistic data is not entered into the system.

These constraints collectively help maintain the accuracy, consistency, and reliability of the database.

3.4 Sample Data

Sample data was inserted into each table to test the system:

- **Customers:** Includes example people with different contact details.
- **Rooms:** Includes single, double, and suite rooms with prices and availability.
- **Bookings:** Shows which customer booked which room on specific dates.
- **Payments:** Records the corresponding payment for each booking.

Using this sample data helps ensure that the database works correctly, relationships between tables function as expected, and queries return accurate results. It also demonstrates how the system handles real-world hotel booking operations.

4. Results

These were the results obtained during the system development and included are the screenshots of them:

- Insert queries.

I created 4 tables: Customers', Rooms', Bookings' and Payments' Tables and inserted 2 customers in the system with their respective identities, allocated to their specific rooms along with the room category, its pricing, the method of payments and the specific dates reserved of stay in the hotel.

hotel_management

Limit to 1000 rows

```sql
INSERT INTO Customers VALUES
(1, 'Alice Brown', 'alice@email.com', '111111111'),
(2, 'David Green', 'david@email.com', '222222222');

INSERT INTO Rooms VALUES
(201, 'Single', 60.00, 'Available'),
(202, 'Double', 90.00, 'Available');

INSERT INTO Bookings VALUES
(1, 1, 201, '2026-04-15', '2026-04-18');

INSERT INTO Payments VALUES
(1, 1, 180.00, '2026-04-14', 'Card');
```

- Update queries.

The Update Query is used to change the status of something specific for example: Room 201 changes now from being available to being Booked and therefore cannot be accessed by another individual.

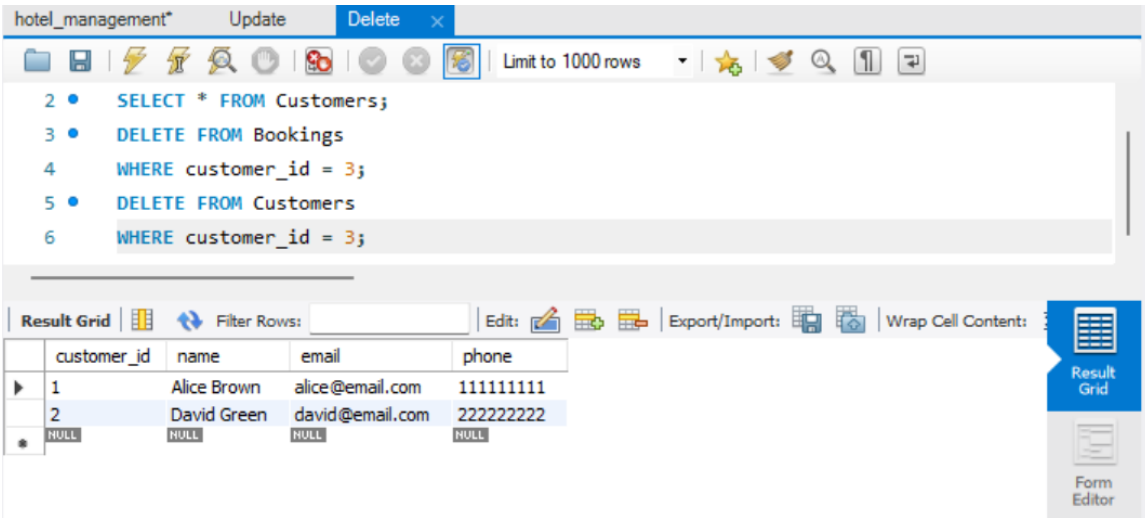
hotel_management* Update

Limit to 1000 rows

```sql
USE hotel_management_db;
UPDATE Rooms
SET status = 'Booked'
WHERE room_id = 201;
```

- Delete queries.

The Delete Query operation removes an individual from the system to maintain database integrity and in order to avoid misconduct of the system by first removing the individual's booking spot to avoid foreign key errors and then remove the individual from the customer's table list.



- JOIN queries.

The JOIN query is used to combine information from multiple tables to provide a comprehensive view of all bookings. By linking the Bookings table with the Customers and Rooms tables using foreign key relationships, the query retrieves each customer's name, the type of room they booked, and the corresponding check-in and check-out dates. This allows hotel management to see all active bookings in a single view, making it easier to track which customers have reserved which rooms and for what duration making it convenient.

```
-- JOIN
SELECT c.name, r.room_type, b.check_in_date
FROM Bookings b
JOIN Customers c ON b.customer_id = c.customer_id
JOIN Rooms r ON b.room_id = r.room_id;
```

- Aggregation queries.

This query calculates the total revenue generated from all payments in the system. The output shows the sum of the amount column from the Payments table. Aggregation queries like this help the hotel management quickly assess overall income.

```sql

-- Aggregation
SELECT SUM(amount) AS total_revenue FROM Payments;

```

- Filtering queries.

This query retrieves all rooms that are currently available for booking. Filtering queries allow the management to quickly find available rooms based on certain criteria, such as status, room_type, or price.

```sql

-- Filtering
SELECT * FROM Rooms WHERE status = 'Available';
```

5. Challenges and Solutions

During the development of the hotel booking system, several challenges were encountered:

1. **Managing Data Relationships:**
   Ensuring that customers, bookings, rooms, and payments were properly linked was initially challenging.
   Solution: Careful design of foreign keys and table relationships in the ER diagram ensured that all data connections were accurate.

2. **Avoiding Data Redundancy:**
   Duplicate information, especially in customer and room details, could cause inconsistencies.
   Solution: The database was normalized to 3NF, which reduced redundancy and improved efficiency.

3. **Handling Updates and Deletions:**
   Updating or deleting data without affecting related records required careful planning.
   Solution: Cascading rules and structured SQL queries were used to safely update and delete data while maintaining integrity.

4. **Testing Queries:**
   Writing and testing complex queries like joins, filters, and aggregations was sometimes tricky.
   Solution: Sample data was used to test all queries thoroughly, ensuring that outputs were correct and meaningful.

6. Conclusion and Future Work

6.1 Conclusion

This project successfully developed a hotel booking database system capable of managing customers, rooms, bookings, and payments. The design followed a structured approach, from creating an Entity-Relationship diagram to implementing tables with appropriate primary and foreign keys to ensure data integrity. Normalization was applied to reduce redundancy and improve efficiency. A range of SQL queries, including inserts, updates, deletes, joins, aggregations, and filters, demonstrated the system's functionality and confirmed its ability to handle real-world booking scenarios. Overall, the project met its objectives while providing practical experience in database design, implementation, and query development.

6.2 Future Work

In conclusion, while the current hotel booking system effectively manages basic operations, there are several opportunities for enhancement. Implementing a web-based interface would enable online bookings and streamlined data management for administrators. Incorporating user authentication would enhance security and allow customers to access their booking history. Additional features such as customer reviews, advanced analytics for booking trends and revenue, and integration with external payment systems could further improve functionality and provide valuable insights to support informed decision-making. These improvements would make the system more user-friendly, secure, and practical for real-world deployment.